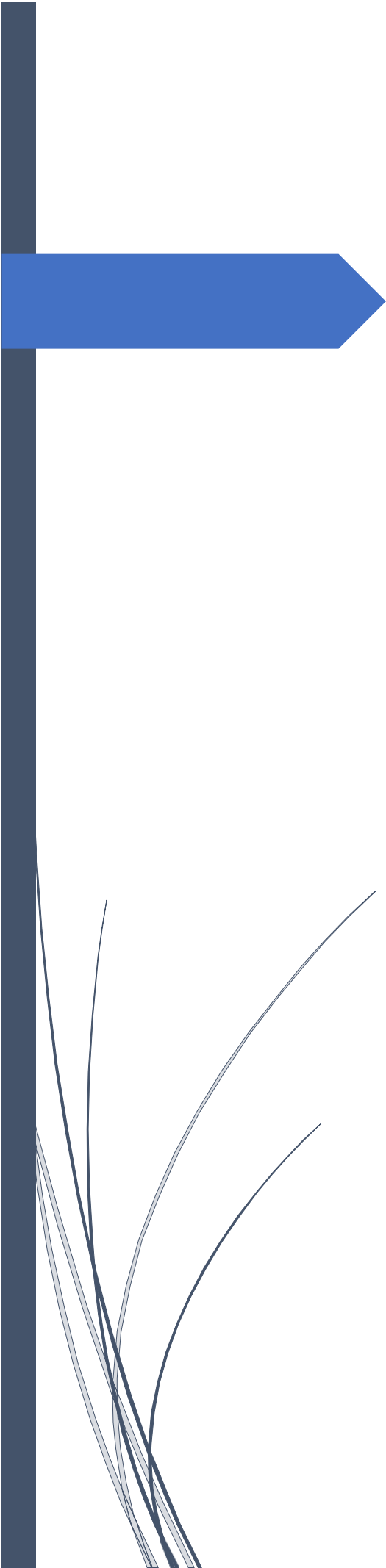




Optimizasyon Yöntemleri Ödevi Raporu

İlke Aşan

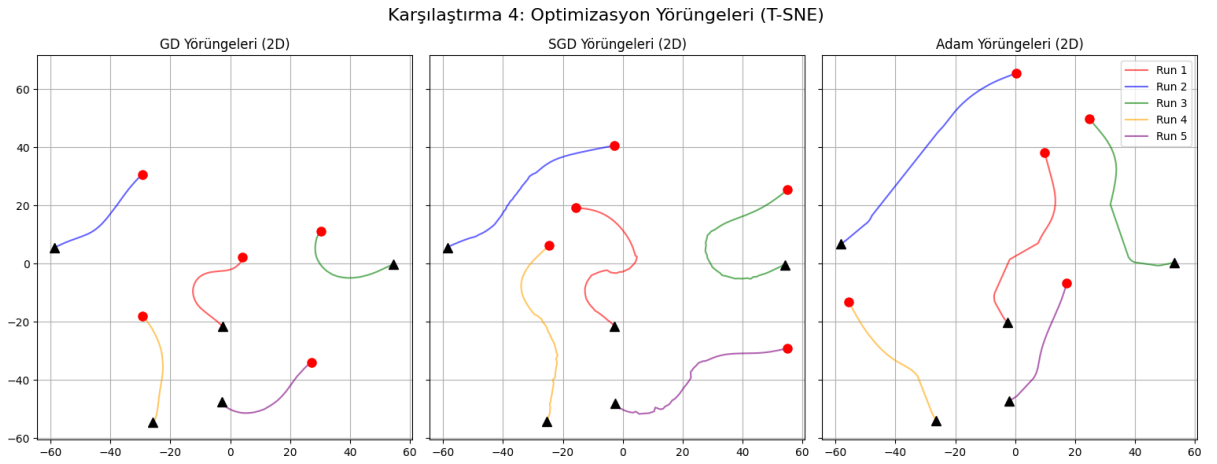
Optimizasyon Yöntemlerinin Karşılaştırılması Raporu

1. Projenin Amacı:

Bu projede bizim oluşturduğumuz test ve eğitim verilerini kullanarak üç farklı algoritmayla (GD, SGD, Adam) modelin w parametresini optimize etmektir. Modelde, girdiler ve w parametresi çarpılarak aktivasyon fonksiyonumuz olan tanh içerisinde geçirilir ve bir tahmin (pred) oluşturulur. Bu tahmin ile gerçek değer arasındaki farktan elde edilen Loss (Hata) fonksiyonu, optimizasyon algoritmalarıyla minimize edilmeye çalışılır.

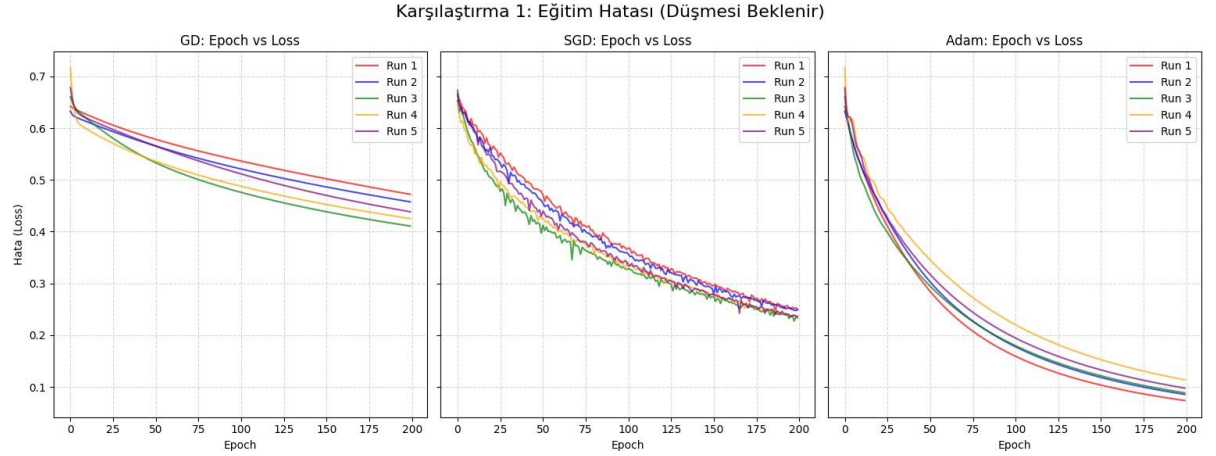
2. Grafik Yorumları

T_SNE



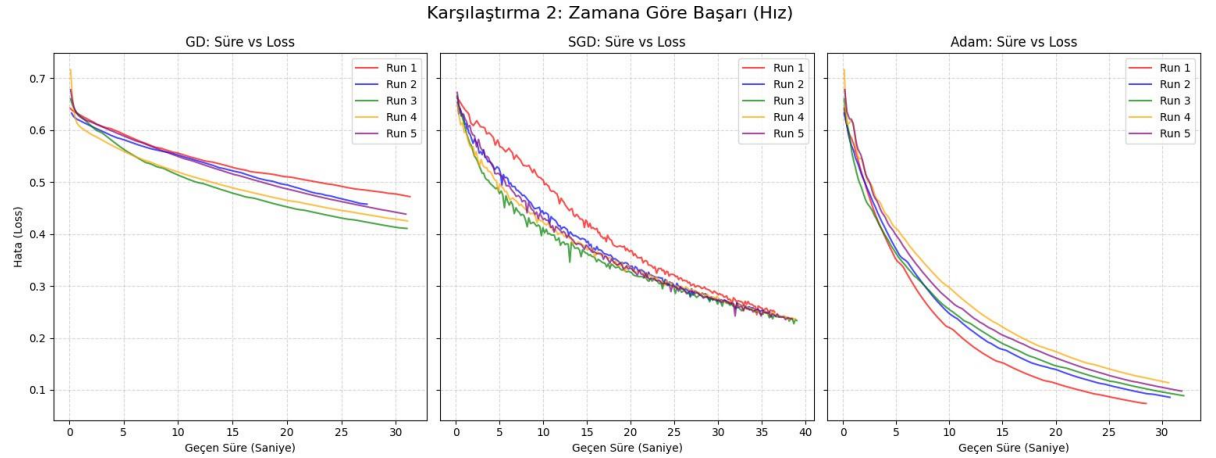
Grafiklerde, 3 farklı algoritmanın aynı başlangıç w değerlerinden yola çıkarak benzer minimum noktalarına yaklaştığı görülmektedir. Ayrıca, farklı başlangıç ağırlıklarının algoritmaları başlangıçta farklı yerel minimum noktalarına yönlendirdiği gözlemlenmiştir. Buna ek olarak; SGD'nin izlediği yoldaki zikzaklı yapı, algoritmanın her adımda farklı bir veri örneği üzerinden optimizasyon yapmasından (stokastik yapı) kaynaklanmaktadır. Adam algoritması ise momentum özelliği sayesinde bu salınımları engelleyerek içlerinde en kararlı ve hızlı yakınsayan yöntem olmuştur.

Epoch Loss:



Elde edilen grafiklerde; SGD ve Adam algoritmaları için 200 epoch süresince yeterli öğrenmenin sağlandığı, ancak GD algoritması için daha fazla epoch sayısına ihtiyaç duyulduğu gözlemlenmiştir. Adam algoritması, momentum özelliği sayesinde bu yöntemler arasında en verimli sonucu vermiştir. Ayrıca, SGD'nin optimizasyon sürecinde her adımda farklı veri örneklerini kullanmasından kaynaklanan karakteristik zikzakları grafiklerde net bir şekilde görülmektedir. Buna karşın GD, daha kararlı ve pürüzsüz bir eğriyle minimum hata (loss) noktasına ulaşmaya çalışmaktadır.

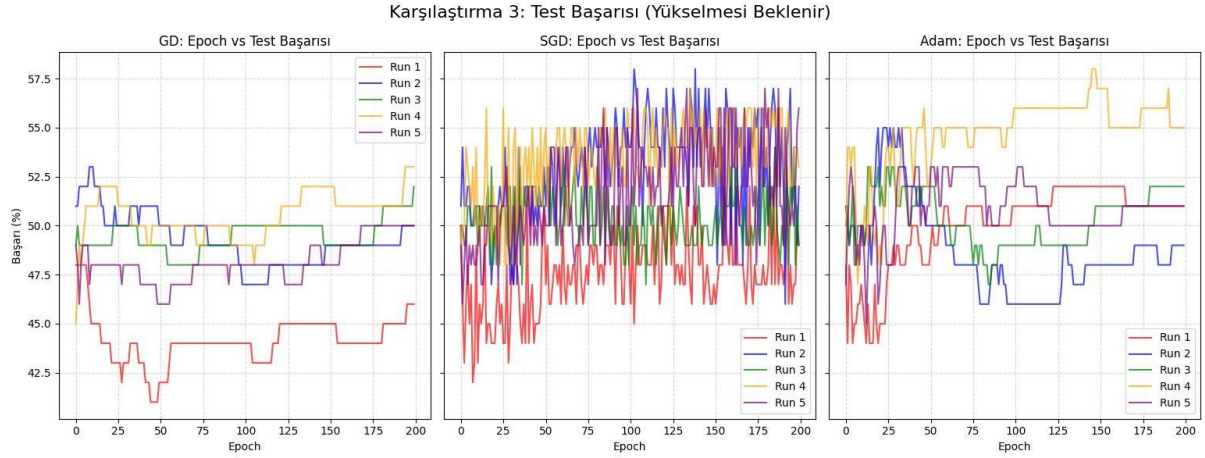
Time Loss:



Grafik sonuçlarında GD algoritması, 200 epoch'u yaklaşık 28 saniyede tamamlayarak işlem hızı bakımından hızlı görünmektedir. Ancak bu sürenin sonunda hata payının (loss) hala 0.4 seviyelerinde kalması, hedefe ulaşma konusunda yavaş kaldığını gösterir. Buna karşın Adam algoritması, neredeyse aynı sürede (29 saniye) hatayı 0.1 seviyesine indirmeyi başarmıştır. Bu

durum, Adam algoritmasının işlem süresi benzer olsa bile öğrenme hızı bakımından çok daha üstün olduğunu kanıtlamaktadır

Test Başarısı :



Loss başarıyla düşmesine rağmen test başarısının %50 civarında sabitlendiği görülmektedir. Veri setimiz dengeli (%50 Doğru, %50 Yanlış etiketli) bir yapıdadır. Bu durumda %50 başarı oranı, modelin aslında veriyi genelleştiremediğini ve rastgele tahmin yaptığını gösterir.

Bu da bize modelimizin overfitting olduğunu gösterir. Model, eğitim verisindeki örnekleri ezberleyerek hatayı düşürmüş, ancak hiç görmediği test verisiyle karşılaştığında doğru ayrımı yapamamıştır. Bu durumu çözmek için MLP (çok katmanlı yapı) kullanmak daha karmaşık örüntülerin öğrenilebilmesini sağlayabilir .

Kodlar:

1. VEKTÖR İŞLEMLERİ

```
def vektor_carpimi(v1, v2):
    toplam = 0.0
    for i in range(len(v1)):
        toplam += v1[i] * v2[i]
    return toplam

def vektor_toplama(v1, v2):
    yeni = []
    for i in range(len(v1)):
        yeni.append(v1[i] + v2[i])
    return yeni

def vektor_cikarma(v1, v2):
    yeni = []
    for i in range(len(v1)):
        yeni.append(v1[i] - v2[i])
    return yeni

def sayi_carpma(vektor, sayi):
    yeni = []
    for eleman in vektor:
        yeni.append(eleman * sayi)
    return yeni

# --- Adam için ---
def vektor_eleman_carpimi(v1, v2):
    yeni = []
    for i in range(len(v1)):
        yeni.append(v1[i] * v2[i])
    return yeni
```

Kodun ilk bölümü gerekli olan vektörel işlem fonksiyonlarının tanımlanmasıyla başlar bu kısımda elimizdeki vektör verilerinin boyutları belli olduğundan v1 v2 arraylerinin eşit boyutlu olduğu düşünülerek fonksiyonlar tanımlanmıştır.

3. TEST VERİLEİ BAŞARISI KONTROLÜ

```
# --- BAŞARI HESAPLAMA (ACCURACY) ---
def skor_hesapla(w, X_data, Y_data):
    dogru = 0
    for i in range(len(X_data)):
        z = vektor_carpimi(w, X_data[i])
        pred = math.tanh(z)
        # +1 veya -1 kararı
        tahmin = 1.0 if pred >= 0 else -1.0
        if tahmin == Y_data[i]:
            dogru += 1
    return (dogru / len(X_data)) * 100
```

Bu kısım test verilerini ve optimize edilmiş w değerini kullanarak pred (tahmin) değerini hesaplayıp Y değeriyle [gerçek tahmin] karşılaştırır Modelin çıktısı işaret kontrolüne tabi tutularak; pozitif sonuçlar +1, negatif sonuçlar -1 olarak sınıflandır ve elde edilen bu tahminler ile gerçek değerlerin eşleşme oranı hesaplanarak, algoritmaların öğrenme başarısı belirlenir.

4. VERİLERİ YÜKLEME

```
def veri_yukle(dosya_adi):
    print(f"'{dosya_adi}' aranıyor...")
    try:
        with open(dosya_adi, 'r', encoding='utf-8') as f:
            veri = json.load(f)
            X = [d['x'] for d in veri]
            Y = [d['y'] for d in veri]
            print(f"Başarıyla! {len(X)} satır veri yüklendi.")
            return X, Y
    except FileNotFoundError:
        print(f"HATA: '{dosya_adi}' bulunamadı!")
        return [], []
```

Bu fonksiyon, .JSON formatında saklanan eğitim ve test verilerini okuyarak X ve Y eğitim ve test belleklerine aktarır. Try-except bloğu kullanılarak, dosyanın bulunamaması gibi durumlarda bizi uyarır.

5. GD ALGORİTMASI

```
def train_gd(X, Y, X_test, Y_test, epochs, lr, w_init, deneme_no):
    w = copy.deepcopy(w_init)
    N = len(X)
    dim = len(w)

    loss_hist = []
    time_hist = []
    acc_hist = [] # Test Başarısı
    w_hist = []

    start_time = time.time()
    print(f" [GD - Run {deneme_no}] Başlıyor...")

    for epoch in range(epochs):
        toplam_hata_karesi = 0.0
        grad_sum = [0.0] * dim

        for i in range(N):
            x = X[i]; y_true = Y[i]
            z = vektor_carpimi(w, x); pred = math.tanh(z)
            err = pred - y_true; toplam_hata_karesi += err**2

            d_tanh = 1.0 - pred**2; grad_factor = err * d_tanh
            grad_i = sayi_carpma(x, grad_factor)
            grad_sum = vektor_toplama(grad_sum, grad_i)

        avg_grad = sayi_carpma(grad_sum, 1.0/N)
        update = sayi_carpma(avg_grad, lr)
        w = vektor_cikarma(w, update)
```

Algoritmanın temel çalışma prensibi, ağırlık parametrelerini (w) güncellemek için her epoch ta tüm eğitim veri setini taramasıdır.

İç döngüde (for i in range(N)), her bir veri noktası için tahmin yapılır ve hata hesaplanır. Ancak ağırlıklar anında güncellenmez; bunun yerine hesaplanan gradyanlar grad_sum değişkeninde toplanır.

Daha sonra bu toplam gradyanın veri sayısına (N) bölünmesiyle avg_grad elde edilir .

Her epoch un sonunda hesaplanan bu ortalama gradyan ve learning rate kullanılarak yalnızca bir kez güncellenir.

```

# Kayıtlar
loss = (toplam_hata_karesi / 2) / N
current_time = time.time() - start_time

# HER TURDA TEST BAŞARISI ÖLÇÜMÜ
acc = skor_hesapla(w, X_test, Y_test)

loss_hist.append(loss)
time_hist.append(current_time)
acc_hist.append(acc)
w_hist.append(copy.deepcopy(w))

if (epoch + 1) % 20 == 0:
    print(f"    Epoch {epoch+1:3d} | Loss: {loss:.4f} | Test Acc: {acc:.1f}% | Süre: {current_time:.2f}s")

return loss_hist, time_hist, acc_hist, w_hist

```

Bu kısım zaman loss değeri (negatif ve pozitif değerlerin birbirini götürmemesi için karesi alınarak N veri sayısına bölünerek hesaplanmıştır) başarı ve w değerlerinin kayıtlarının yapıldığı kısımdır son bölümde ise konsola her 20 epoch ta bu kayıtlar kontrol sağlanabilmesi için yazdırılır.

6. SGD

```

def train_sgd(X, Y, X_test, Y_test, epochs, lr, w_init, deneme_no):
    w = copy.deepcopy(w_init)
    N = len(X)
    loss_hist = []
    time_hist = []
    acc_hist = []
    w_hist = []

    start_time = time.time()
    print(f"    [SGD - Run {deneme_no}] Başlıyor...")
    indices = list(range(N))

    for epoch in range(epochs):
        random.shuffle(indices)
        toplam_hata_karesi = 0.0

        for i in indices:
            x = X[i]; y_true = Y[i]
            z = vektor_carpimi(w, x); pred = math.tanh(z)
            err = pred - y_true; toplam_hata_karesi += err**2

            d_tanh = 1.0 - pred**2; grad_factor = err * d_tanh
            grad_i = sayi_carpma(x, grad_factor)

            update = sayi_carpma(grad_i, lr)
            w = vektor_cikarma(w, update)

        loss = (toplam_hata_karesi / 2) / N
        current_time = time.time() - start_time

    # Test Başarısı
    acc = skor_hesapla(w, X_test, Y_test)

    loss_hist.append(loss)
    time_hist.append(current_time)
    acc_hist.append(acc)
    w_hist.append(copy.deepcopy(w))

    if (epoch + 1) % 20 == 0:
        print(f"    Epoch {epoch+1:3d} | Loss: {loss:.4f} | Test Acc: {acc:.1f}% | Süre: {current_time:.2f}s")

    return loss_hist, time_hist, acc_hist, w_hist

```

Sgd fonksiyonunda GD den farklı olarak her turda farklı X_train verisiyle optimizasyon yapar. Kodumda her seferinde farklı X değerini kullanmasını garantilemek için rondon.shuffle() fonksiyonuyla X verilerini karıştırdım ve bu karışan verileri for döngüsünde sırayla kullandım.

İç döngüdeki tanh fonksiyonuyla pred (tahmin) ve gradyan hesabi işlemleri GD ile benzerdir.

Son kısımda ise yine GD ile benzer olarak kayıtlar tutulmuştur.

7. ADAM:

```
def train_adam(X, Y, X_test, Y_test, epochs, lr, w_init, deneme_no, beta1=0.9, beta2=0.999, eps=1e-8):
    w = copy.deepcopy(w_init)
    N = len(X)
    dim = len(w)
    m = [0.0] * dim; v = [0.0] * dim

    loss_hist = []
    time_hist = []
    acc_hist = []
    w_hist = []

    start_time = time.time()
    print(f" [Adam - Run {deneme_no}] Başlıyor...")

    t = 0
    for epoch in range(epochs):
        t += 1
        toplam_hata_karesi = 0.0
        grad_sum = [0.0] * dim

        for i in range(N):
            x = X[i]; y_true = Y[i]
            z = vektor_carpimi(w, x); pred = math.tanh(z)
            err = pred - y_true; toplam_hata_karesi += err**2

            d_tanh = 1.0 - pred**2; grad_factor = err * d_tanh
            grad_i = sayi_carpma(x, grad_factor)
            grad_sum = vektor_toplama(grad_sum, grad_i)

        g = sayi_carpma(grad_sum, 1.0/N)

        m_part1 = sayi_carpma(m, beta1); m_part2 = sayi_carpma(g, (1 - beta1))
        m = vektor_toplama(m_part1, m_part2)

        g_kare = vektor_eleman_carpimi(g, g)
        v_part1 = sayi_carpma(v, beta2); v_part2 = sayi_carpma(g_kare, (1 - beta2))
        v = vektor_toplama(v_part1, v_part2)

        m_hat = sayi_carpma(m, 1.0 / (1 - beta1**t))
        v_hat = sayi_carpma(v, 1.0 / (1 - beta2**t))

        update_term = vektor_bolme_ozel(m_hat, v_hat, eps)
        step = sayi_carpma(update_term, lr)
        w = vektor_cikarma(w, step)

        loss = (toplam_hata_karesi / 2) / N
        current_time = time.time() - start_time

        # Test Başarısı
        acc = skor_hesapla(w, X_test, Y_test)

        loss_hist.append(loss)
        time_hist.append(current_time)
        acc_hist.append(acc)
        w_hist.append(copy.deepcopy(w))

        if (epoch + 1) % 20 == 0:
            print(f" Epoch {epoch+1:3d} | Loss: {loss:.4f} | Test Acc: {acc:.1f}% | Süre: {current_time:.2f}s")

    return loss_hist, time_hist, acc_hist, w_hist
```

Kodun iç döngüsünde, öncelikle diğer algoritmalarındaki gibi gradyan faktörü hesaplanır. Daha sonra Adam algoritmasının momentum (m) ve hız (v) değerleri belirlenir. m_part1 kısmında, beta1 katsayısı ile çarpma yapılarak geçmiş verilerin momentumunun sonuca ne kadar etki edeceği belirlenir. m_part2 kısmında ise şu anki gradyanın etkisi hesaplanır ve bu iki değer toplanarak m momentumu güncellenir. Benzer bir ağırlıklı ortalama işlemi v (hız) değerinin hesabında da gerçekleştirilir. Son aşamada, update_term kısmında her bir parametre için özel adım büyüklüğü hesaplanır ve bu değer kullanılarak w ağırlıkları optimize edilir

8. FONKSİYONLARIN ÇALIŞTIRILMASI

```
X_train, Y_train = veri_yukle("egitim_vektorleri.json")
X_test, Y_test = veri_yukle("test_vektorleri.json")

if len(X_train) == 0 or len(X_test) == 0:
    exit()

input_dim = len(X_train[0])
EPOCH = 200
LR = 0.5

sonuclar = {'GD': [], 'SGD': [], 'Adam': []}

print(f"\n[EPOCH] Epochluk 5 Farklı Deney Başlıyor...")

baslangic_w_listesi = []
for i in range(5):
    w_random = [random.uniform(-1.0, 1.0) for _ in range(input_dim)]
    baslangic_w_listesi.append(w_random)

for i in range(5):
    print(f"=== DENEME {i+1}/5 ===")
    w_sabit = baslangic_w_listesi[i]

    # Eğitim ve Test
    l_gd, t_gd, acc_gd, w_gd = train_gd(X_train, Y_train, X_test, Y_test, EPOCH, LR, w_sabit, i+1)
    sonuclar['GD'].append({'loss': l_gd, 'time': t_gd, 'acc': acc_gd, 'w': w_gd})
    print("")

    l_sgd, t_sgd, acc_sgd, w_sgd = train_sgd(X_train, Y_train, X_test, Y_test, EPOCH, LR*0.05, w_sabit, i+1)
    sonuclar['SGD'].append({'loss': l_sgd, 'time': t_sgd, 'acc': acc_sgd, 'w': w_sgd})
    print("")

    l_adam, t_adam, acc_adam, w_adam = train_adam(X_train, Y_train, X_test, Y_test, EPOCH, 0.01, w_sabit, i+1)
    sonuclar['Adam'].append({'loss': l_adam, 'time': t_adam, 'acc': acc_adam, 'w': w_adam})
    print("-" * 40)

print("\nSonuçlar Bittii Grafikler Çiziliyor...")
```

Bu kısımda 5 turluk bir döngü kuruyoruz. Her turun başında rastgele bir w başlangıç değeri üretiyoruz ve bu değeri kopyalayarak hem GD'ye, hem SGD'ye hem de Adam'a başlangıç noktası olarak veriyoruz.

9. Grafiklerin çizilmesi:

```
# --- GRAFİK 1: EPOCH vs. LOSS (Eğitim Hatası) ---
fig1, ax1 = plt.subplots(1, 3, figsize=(18, 6), sharey=True)
for idx, algo in enumerate(algoritmalar):
    for run in range(5):
        ax1[idx].plot(sonuclar[algo][run]['loss'], color=renkler[run], alpha=0.7, label=f"Run {run+1}")
    ax1[idx].set_title(f"{algo}: Epoch vs Loss")
    ax1[idx].set_xlabel("Epoch")
    if idx==0: ax1[idx].set_ylabel("Hata (Loss)")
    ax1[idx].grid(True, linestyle='--', alpha=0.5)
    ax1[idx].legend()
plt.suptitle("Karşılaştırma 1: Eğitim Hatası", fontsize=16)
plt.tight_layout()
plt.show()

print("T-SNE hesaplanıyor...")
dev_w_listesi = []
for algo in algoritmalar:
    for run in range(5):
        dev_w_listesi.extend(sonuclar[algo][run]['w'])

tsne = TSNE(n_components=2, random_state=42, perplexity=30)
w_2d = tsne.fit_transform(np.array(dev_w_listesi))

fig4, ax4 = plt.subplots(1, 3, figsize=(18, 6), sharex=True, sharey=True)
current_idx = 0
for idx, algo in enumerate(algoritmalar):
    ax4[idx].set_title(f"{algo} Yörüngeleri (2D)")
    for run in range(5):
        uzunluk = len(sonuclar[algo][run]['w'])
        yol = w_2d[current_idx : current_idx + uzunluk]
        current_idx += uzunluk
        ax4[idx].plot(yol[:,0], yol[:,1], marker='', linestyle='--', color=renkler[run], alpha=0.6, label=f"Run {run+1}")
        ax4[idx].plot(yol[0,0], yol[0,1], 'k^', markersize=8)
        ax4[idx].plot(yol[-1,0], yol[-1,1], 'ro', markersize=8)
    ax4[idx].grid(True)
    if idx==2: ax4[idx].legend()

plt.suptitle("Karşılaştırma 4: Optimizasyon Yörüngeleri (T-SNE)", fontsize=16)
plt.tight_layout()
plt.show()
```